Evaluating and Refining Arch-Stage

Supporting Architectural Hypotheses in Software Engineering Teams

José Gonçalves Lima Neto

Thesis presented to the
Institute of Mathematics,
Statistics, and Computer Science
of the University of São Paulo
in partial fulfillment
of the requirements
for the degree of
Master of Science

Program: Computer Science
Advisor: Prof. Dr. Paulo Roberto Miranda Meirelles

São Paulo
December, 2026

Evaluating and Refining Arch-Stage

Supporting Architectural Hypotheses in Software Engineering Teams

José Gonçalves Lima Neto

This is the original version of the thesis prepared by candidate José Gonçalves Lima Neto, as submitted to the Examining Committee.

The content of this work is published under the CC BY 4.0 license
(Creative Commons Attribution 4.0 International License)

Acknowledgments

Abstract

José Gonçalves Lima Neto. **Evaluating and Refining Arch-Stage: *Supporting Architectural Hypotheses in Software Engineering Teams***. Thesis (Master's). Institute of Mathematics, Statistics, and Computer Science, University of São Paulo, São Paulo, 2026.

Software engineering teams frequently face uncertainties related to their systems' architecture yet lack adequate methods and tools to manage them explicitly and collaboratively. The ArchHypo technique proposes using hypotheses engineering to make such uncertainties visible and manageable; however, its manual application imposes a significant learning curve. This work empirically evaluates and refines Arch-Stage, a support tool for the ArchHypo technique, by applying it with multiple software engineering teams in a real-world organization with a scaled software product. The research adopts a mixed-methods design combining analysis of artifacts produced in the tool, questionnaires, and interviews with team members. Preliminary results indicate how the tool supports the management of architectural uncertainties and inform its ongoing improvement.

Keywords: Software Architecture. Architectural Uncertainty. Hypotheses Engineering. ArchHypo. Arch-Stage. Empirical Evaluation.

Resumo

José Gonçalves Lima Neto. **Avaliação e Refinamento do Arch-Stage:** ***Apoio a Hipóteses Arquiteturais em Equipes de Engenharia de Software***. Dissertação (Mestrado). Instituto de Matemática, Estatística e Ciência da Computação, Universidade de São Paulo, São Paulo, 2026.

Equipes de engenharia de software frequentemente enfrentam incertezas relacionadas à arquitetura de seus sistemas, mas carecem de métodos e ferramentas adequadas para gerenciá-las de forma explícita e colaborativa. A técnica ArchHypo propõe o uso de engenharia de hipóteses para tornar essas incertezas visíveis e gerenciáveis, porém sua aplicação manual impõe uma curva de aprendizado significativa. Este trabalho avalia e refina empiricamente o Arch-Stage, uma ferramenta de apoio à técnica ArchHypo, por meio de sua aplicação com múltiplas equipes de engenharia de software em uma organização real com um produto em escala. A pesquisa adota um design de métodos mistos, combinando análise de artefatos produzidos na ferramenta, questionários e entrevistas com membros das equipes. Os resultados preliminares indicam como a ferramenta apoia o gerenciamento de incertezas arquiteturais e informam seu aprimoramento contínuo.

Palavras-chave: Arquitetura de Software. Incerteza Arquitetural. Engenharia de Hipóteses. ArchHypo. Arch-Stage. Avaliação Empírica.

List of Abbreviations

SA	Software Architecture
ArchHypo	Architecture Hypothesis (technique)
IDP	Internal Developer Portal
NFR	Non-Functional Requirements
QA	Quality Attribute
DSR	Design Science Research
HE	Hypotheses Engineering
IME	Institute of Mathematics and Statistics (Universidade de São Paulo)
USP	University of São Paulo
ADR	Architecture Decision Record

List of Figures

4.1 Conceptual framework of the study. 25
4.2 Overview of the ArchHypo technique showing the iterative cycle through which engineers identify architectural hypotheses, assess their uncertainty and impact, define technical plans, and take or postpone architectural decisions. Source: adapted from K. Silva, J. Melegati, *et al.*, 2025. 27

List of Tables

List of Programs

Contents

1 Introduction **1**

1.1 Architectural Uncertainty and Hypotheses Engineering 2
1.2 From Requirements to Architecture and the Role of Tool Support 3
1.3 Problem Statement 4
1.4 Objectives 4
1.5 Research Questions 5
1.6 Research Contributions 5
1.7 Research Structure 6

2 Background **7**

2.1 Software Architecture and Architectural Uncertainty 7
2.2 Hypotheses Engineering and ArchHypo 8
2.2.1 Core Concepts 8
2.2.2 Assessment 9
2.2.3 Technical Plans 9
2.2.4 Lifecycle 10
2.2.5 Empirical Evaluations 10
2.3 Internal Developer Portals and Backstage 12
2.3.1 The Role of Internal Developer Portals (IDPs) 12
2.3.2 Backstage: Open Platform for Internal Developer Portals 12
2.4 Arch-Stage: The Initial Tool 13
2.5 Deriving Architecture from Requirements: The Tooling Gap 14

3 Related Work **17**

3.1 Centralization and Decentralization of Architectural Decisions 17
3.2 Socio-Technical Architecture 18
3.2.1 Conway's Law and Its Implications 18
3.2.2 Socio-Technical Architecture as an Industry Trend 19

3.2.3 Centralization vs. Decentralization: The Core Tension 19
3.3 Tool Support for Architectural Decision-Making 20
3.4 Empirical Studies of Architectural Practices in Industry 21

4 Research Design **23**
4.1 Research Strategy 23
4.2 Conceptual Research Framework 24
4.3 Research Context 25
4.3.1 Case Selection 25
4.4 Researcher's Role 26
4.5 Procedure 27
4.5.1 Training and Introduction 27
4.5.2 Observation Period 27
4.6 Data Collection 28
4.6.1 Participant Observation 28
4.6.2 Artifact Analysis 29
4.6.3 Open-Ended Active Listening Sessions 29
4.7 Data Analysis 29
4.8 Establishing Trustworthiness 30
4.9 Ethical Considerations 30

5 Results **33**

6 Refinements **35**

7 Conclusion **37**

Appendixes

A Research Instruments **39**
A.1 Questionnaire 39
A.2 Interview Guide 39
A.3 Informed Consent Form 39
A.4 Training Communication 39

References **41**

Chapter 1

Introduction

Software architecture (SA) plays a central role in the development and evolution of software-intensive systems. As firstly conceived by Perry and Wolf, 1992, the SA concept structures the system into elements and relations that form the basis of the system's organization as it must convey cohenrence, scalability, and maintainability for the relaying software over time. Despite the ever-changing practices in the modern era of software development, SA still provides the basis for analyzing the multiple quality attributes of nowadays software systems as it guides technical decisions that must be made throughout the development process over time via a diverse set of archictetual methodologies as the ones systematically mapped by Souza *et al.*, 2019. In organizations that rely on software as a core part of their business, architecture is not just a simple one-time design artifact, but a continuous decision-making process that must accommodate changing requirements, evolving technologies, and critical organizational constraints K. Silva, J. Melegati, *et al.*, 2025.

In recent years, agile and continuous development practices have intensified this fast-paced pressure. Teams are expected to deliver more features in shorter cycles while preserving scalability, performance, and other quality attributes of their systems that continue to grow in size and complexity K. Silva, J. Melegati, *et al.*, 2025; K. Silva, J. A. Melegati, *et al.*, 2024. In such environments, architecture decisions are frequently made under uncertainty: solutions or requirements may be incomplete or unstable, the impact of technology choices may not be fully understood, and the behavior of the system under realistic workloads may only become visible after deployment K. Silva, J. Melegati, *et al.*, 2025. Despite this, architectural uncertainties often remain implicit and unmanaged, and decisions tend to rely on the tacit knowledge and intuition of a small group of experienced architects as oberseved by Souza *et al.*, 2019.

At the same time, many organizations seek to organize themselves around autonomous, cross-functional teams responsible for end-to-end delivery of features. Recent industry reports, such as the 2024 InfoQ Software Architecture and Design Trends Report Betts *et al.*, 2024, highlight socio-technical architecture as an emerging trend that emphasizes the co-design of technical and organizational structures, and the importance of considering how team boundaries and communication patterns shape system architectures. In this view,

architecture work is distributed across teams rather than centralized in a separate function or role. Moreover, in these scenarios, the Conway's Law is treated as a design constraint to be intentionally leveraged rather than an accident to be ignored Betts *et al.*, 2024. However, when architecture decisions depend on implicit assumptions held by a few experts in the organization, this distribution becomes difficult in practice. Less experienced team members may struggle to participate meaningfully in architectural discussions, and teams may find it hard to reason about the risks and trade-offs of alternative solutions even when they are applying anyone of thewell-known and validated architectural methodologies mapped by Souza *et al.*, 2019. This challenge reveals the case of the architectural uncertainty management, which still is a critical aspect of the architecture work, but is often neglected or poorly supported in practice by the available tools and methodologies.

1.1 Architectural Uncertainty and Hypotheses Engineering

Uncertainty in architecture decisions stems from limited knowledge about requirements, future usage scenarios, and the properties of candidate solutions as reviewed in K. Silva, J. Melegati, *et al.*, 2025. Traditional risk management techniques assume that relevant events and their probabilities are known or can be estimated, which is not always the case when dealing with evolving systems and emerging technologies K. Silva, J. Melegati, *et al.*, 2025. A complementary perspective is to acknowledge uncertainty explicitly and as an inevitable aspect of the architecture work, and therefore, to manage it as a first-class concern during the architecture work as proposed in K. Silva, J. Melegati, *et al.*, 2025.

In this regard, Hypotheses Engineering has emerged as an approach to manage uncertainty in software development by treating assumptions as explicit, falsifiable hypotheses that can be prioritized and tested through experiments K. Silva, J. Melegati, *et al.*, 2025; K. Silva, J. A. Melegati, *et al.*, 2024. In this context, ArchHypo is the technique that adapts hypotheses engineering to software architecture. It proposes that teams identify *architectural hypotheses* — statements capturing uncertainties that may affect architectural decisions — assess each hypothesis by its uncertainty level and impact, and define a technical plan for handling it K. Silva, J. Melegati, *et al.*, 2025.

The technical plan may include actions to reduce uncertainty, such as experiments, architectural spikes, or tracer bullets, and actions to reduce the impact of potential changes, such as modularization or flexible deployment strategies K. Silva, J. Melegati, *et al.*, 2025. By iteratively updating hypotheses and technical plans over time, ArchHypo aims to support a more deliberate and distributed approach to architectural decision-making, allowing teams to postpone irreversible decisions when justified and to focus their efforts on the most critical uncertainties K. Silva, J. Melegati, *et al.*, 2025; K. Silva, J. A. Melegati, *et al.*, 2024.

Empirical studies have started to explore how ArchHypo works in practice. An experience report in a software start-up describes how the technique helped the founder identify and prioritize architectural uncertainties related to availability, usability, and scalability over a ten-month period, guiding the evolution of the product and avoiding premature investments in infrastructure K. Silva, J. A. Melegati, *et al.*, 2024. A design science

study in a mission-critical public-sector system shows that integrating ArchHypo into the development process enabled the team to distribute architectural work across iterations and to make better-informed decisions in the face of changing requirements K. Silva, J. Melegati, *et al.*, 2025. At the same time, both studies report that the technique introduces a non-trivial learning curve and requires adjustments to existing processes, which might pose a challenge to its adoption by the team as reported in K. Silva, J. Melegati, *et al.*, 2025; K. Silva, J. A. Melegati, *et al.*, 2024.

1.2 From Requirements to Architecture and the Role of Tool Support

The challenges of managing architectural uncertainty are closely related to the long-standing problem of deriving architectural models from requirements specifications. A systematic mapping study by Souza *et al.*, 2019 reviewed 39 methods for this derivation and concluded that existing approaches still rely heavily on the tacit knowledge of architects, provide limited support for less experienced practitioners, and often lack explicit evaluation mechanisms. Many methods describe high-level processes or frameworks, but offer little guidance on how to operationalize them in everyday practice as reported in Souza *et al.*, 2019.

The same study highlighted gaps in tool support. A significant portion of the methods did not include any supporting tool, and when tools were present, they tended to address only parts of the process as also reported in Souza *et al.*, 2019. The authors called for more automation, better mechanisms to capture and reuse architectural knowledge, and empirical validation of tools in realistic settings. These findings resonate with the observations from ArchHypo studies: while the technique provides a structured way to make architectural uncertainties explicit and to plan actions, applying it in practice is highly demanding without adequate supportting tool.

Grounded in these studies' insights, Corrêa, 2025 developed the Arch-Stage, a support tool designed to help teams apply the ArchHypo technique in practice. Up to a specific point in its history, the work focused on designing and implementing the core functionality for enabling ArchHypo capabilities, such as representing architectural hypotheses, recording their uncertainty and impact assessments, and managing associated technical plans.[1] The work also discussed initial design decisions and limitations, including usability aspects and open questions about how the tool would fit into different team workflows. A more detailed description of Arch-Stage and the contributions by Corrêa, 2025 are presented in Chapter ??.

[1] The first version of Arch-Stage and the corresponding source code history up to commit `ceee509776508081ebdd473c2c4f710b8ef55947` document the initial design and implementation of Arch-Stage proposed by Corrêa, 2025.

1.3 Problem Statement

When reviewed in intersection, these studies outline a clear problem. On the one hand, modern software organizations need to manage architectural uncertainty in a way that supports continuous evolution and involves the broader development team K. Silva, J. Melegati, *et al.*, 2025; K. Silva, J. A. Melegati, *et al.*, 2024; Betts *et al.*, 2024. On the other hand, empirical evidence and systematic mapping studies indicate that:

- Methods for deriving and evolving architectures from requirements still depend strongly on architects' tacit knowledge and lack comprehensive tool support as reported in Souza *et al.*, 2019.
- ArchHypo offers a promising structure for handling architectural uncertainty, but teams perceive the technique as hard to learn and requiring process changes, especially without specialized support as reported in K. Silva, J. Melegati, *et al.*, 2025; K. Silva, J. A. Melegati, *et al.*, 2024.
- Arch-Stage provides an initial implementation of tool support for ArchHypo, but its impact on real software engineering teams and the refinements needed for effective use in industry remain unknown as reported in Corrêa, 2025.

In this context, there is a need to understand how software engineering teams, with diversed roles, tenures and responsibilities, interact with a tool like Arch-Stage when working on their products and services. By forging this path, we seek to gain an understanding of what sorts of architectural uncertainties they manage through, and which aspects of the Arch-Stage tool and its integration into the development process still require refinement. Addressing these concerns can contribute both to the practical adoption of ArchHypo via Arch-Stage tool and to the broader discussion on how to support teams in sharing architectural responsibilities for accelerating adoption of the socio-technical architecture paradigm mapped in Betts *et al.*, 2024.

1.4 Objectives

The overall objective of this present work is to empirically evaluate and refine the Arch-Stage tool, a support tool for the ArchHypo technique from Corrêa, 2025, by applying it with diverse software engineering teams in a real-world organization that maintains a scaled software product in Latin America. More specifically, this work aims to:

- Investigate how software engineering teams appropriate and use Arch-Stage and ArchHypo in their day-to-day activities when working on an existing, scaled product.
- Derive and implement refinements to Arch-Stage and its usage guidelines based on empirical findings, with the goal of improving its suitability for supporting architectural work in teams.
- Characterize the architectural uncertainties that teams make explicit and manage through Arch-Stage, and how these uncertainties evolve over time.
- Identify benefits and challenges perceived by team members regarding the use of

Arch-Stage, including its learning curve, usability, and fit with existing practices.

These objectives deliberately build on, rather than replicate, the previous studies' contributions in the ArchHypo research line. While the Corrêa, 2025 focused on designing and implementing Arch-Stage, this work treats the tool as an existing artifact and concentrates on its evaluation and evolution in an industrial context.

1.5 Research Questions

To achieve the above objectives, this work is guided by the following research questions. At this stage, the exact emphasis between tool interaction and uncertainty patterns is still under refinement; therefore, the questions are formulated to accommodate both perspectives, and later chapters will make their final scope explicit.

RQ1: How do software engineering teams use Arch-Stage and the ArchHypo technique when working on a scaled software product in a real-world organization?

RQ2: Which refinements to Arch-Stage and its usage guidelines emerge from the empirical study, and how do they address the identified challenges?

RQ3: What types of architectural uncertainties do these teams identify, record, and manage using Arch-Stage over the course of their work?

RQ4: What benefits and challenges do team members perceive in using Arch-Stage to support architectural decision-making and uncertainty management?

As the empirical design is detailed in Chapter 4, these questions may be operationalized with different emphases — for example, with RQ1 and RQ3 focusing more strongly on interaction and process, and RQ2 providing a descriptive view of uncertainty patterns — to accommodate the different research questions' emphases.

1.6 Research Contributions

By answering its set of research questions, this work attempts to make the following contributions:

- An empirical account of how multiple software engineering teams in a real organization use a tool-supported implementation of ArchHypo (via Arch-Stage tool) while working on a scaled software product, including observed practices, adaptations, and obstacles as reported in Corrêa, 2025.
- A set of empirically grounded refinements to the Arch-Stage tool and its usage guidelines, aimed at reducing the learning curve, improving usability, and better integrating the technique into team workflows as reported in Corrêa, 2025.
- A characterization of architectural uncertainties managed by these teams, describing their sources, quality attributes, and evolution, as captured through Arch-Stage as reported in Corrêa, 2025.

- Reflections on the implications of these findings for supporting distributed architectural work in agile and continuous development settings, connecting the ArchHypo experience to broader concerns about tacit knowledge and tool support in software architecture as reported in Souza *et al.*, 2019; Betts *et al.*, 2024.

1.7 Research Structure

The remainder of this work is organized as follows.

Chapter **??** presents the background and related work. It introduces key concepts in software architecture and architectural uncertainty, summarizes the ArchHypo technique and previous empirical studies about it, describes the initial design of the Arch-Stage tool and the undergraduate thesis that developed it, and revisits the main findings from systematic studies on deriving architecture from requirements and on tool support in this area by Souza *et al.*, 2019.

Chapter 4 details the research design and methods adopted in this dissertation. It describes the industrial context in which the study was conducted, the participating teams and their product, the data collection instruments and procedures, and the analysis methods used to answer the research questions.

The chapter reporting empirical results presents how teams used Arch-Stage and ArchHypo in practice, the architectural hypotheses and uncertainties they managed, and the perceived benefits and challenges from the perspective of different roles.

The chapter discussing refinements details the changes made to Arch-Stage and its usage guidelines based on empirical findings. It explains the rationale for the changes, illustrates how they address specific issues observed in the field, and reflects on remaining limitations and opportunities for future improvement.

Finally, the conclusion chapter summarizes the main contributions, discusses implications for practitioners and researchers, and outlines directions for future work on tool support for architectural uncertainty management and distributed architecture decision-making.

Chapter 2

Background

This chapter establishes the conceptual foundations for the empirical investigation carried out in this thesis. It begins with a concise overview of software architecture and architectural uncertainty in the context of scaled products, then introduces hypotheses engineering and the ArchHypo technique in full technical detail, describes Internal Developer Portals and their role as the delivery platform for tools such as Arch-Stage, presents the Arch-Stage tool itself, and concludes with a summary of findings from Souza et al.'s mapping study on the gap between architectural derivation methods and practical tool support.

2.1 Software Architecture and Architectural Uncertainty

Software architecture refers to the high-level structure of a software system, encompassing its components, their relationships, and the design principles that guide its evolution Bass *et al.*, 2021. In organizations that maintain scaled products—systems developed and operated by multiple teams—architectural decisions shape not only technical quality attributes such as performance, reliability, and maintainability, but also the ability of teams to work independently and deliver changes at speed Skelton and Pais, 2019.

Richards and Ford define software architecture as the structure of a system combined with the characteristics it must support, the principles and decisions guiding its design, and the design decisions themselves Richards and Ford, 2020. This definition emphasizes that architecture is a living body of reasoning, not a frozen blueprint. Perry and Wolf's earlier tripartite model Perry and Wolf, 1992 decomposed architecture into *elements* (the processing, data, and connecting constituents), *form* (the weighted relationships among those elements), and *rationale* (the set of constraints that justify the structure). Together, these accounts foreground the role of uncertainty: the rationale behind any architectural choice is valid only relative to the information available at the time of the decision, and that information is always incomplete.

Architectural uncertainty arises whenever teams must make structural decisions under

incomplete information. This is a normal condition in continuous-delivery environments where business requirements, user behavior, and technology options are only partially known. When uncertainty is left implicit, teams either defer decisions indefinitely (accumulating technical debt) or make premature commitments that constrain future options. Both outcomes slow delivery and degrade system quality Foote and Yoder, 1999.

The concept of "fast flow," popularized by the Team Topologies framework Skelton and Pais, 2019, emphasizes that organizations should design both their teams and their architecture to enable a continuous, rapid stream of software changes. Achieving fast flow presupposes that teams can make local architectural decisions without waiting for a central architect. This in turn demands that architectural uncertainties be made explicit and that lightweight mechanisms exist for teams to assess, prioritize, and revisit them as evidence accumulates.

2.2 Hypotheses Engineering and ArchHypo

Hypotheses engineering is an approach to managing technical uncertainty by formulating architectural decisions as testable hypotheses rather than settled facts. Instead of asserting that a particular technology or pattern *will* satisfy a quality requirement, teams state that it *might* do so and then design experiments to gather evidence.

The ArchHypo technique, proposed by Silva et al., operationalizes this idea for software architecture E. Silva, J. Melegati, Silveira, *et al.*, 2025. Grounded in Design Science Research (DSR) and evolved from a body of patterns for agile architecture first disseminated at conferences and through corporate training E. Silva, J. Melegati, Wang, *et al.*, 2024, ArchHypo provides a structured vocabulary and a set of practices that fit within iterative development workflows.

2.2.1 Core Concepts

An *architectural hypothesis* is a falsifiable statement that captures an uncertainty relevant to the design of a software architecture. It is important to distinguish the concept from adjacent notions:

- **Hypothesis vs. risk.** A risk involves a known event with estimable probability and impact. An architectural hypothesis, by contrast, stems from a *lack of information* sufficient to make a decision. The distinguishing feature is the epistemic gap, not the probability of an outcome E. Silva, J. Melegati, Silveira, *et al.*, 2025.

- **Hypothesis vs. requirement.** A requirement specifies what the system must do or how well it must do it. A hypothesis may be triggered by a requirement (e.g., a quality-attribute goal that cannot yet be confirmed as achievable), but the hypothesis captures the uncertainty about the *path* to satisfying that requirement, not the requirement itself.

- **Hypothesis vs. architectural decision.** An architectural decision resolves a design choice. A hypothesis *precedes and informs* decisions: it identifies what must be

learned or validated before a decision can responsibly be made E. Silva, J. Melegati, Silveira, *et al.*, 2025.

Hypotheses arise from two principal sources: (a) *requirements*—when quality attribute targets are poorly specified or the team lacks information about whether a given approach can satisfy them; and (b) *solutions*—when the consequences of an existing or candidate architectural solution are unknown.

2.2.2 Assessment

Each hypothesis is evaluated along two independent dimensions:

1. **Uncertainty level.** How far the team is from being able to refute or accept the hypothesis. Crucially, uncertainty level is *not* equivalent to probability. A hypothesis that is almost certainly true and a hypothesis that is almost certainly false both represent *low* uncertainty, because in either case the team already has enough information to act. High uncertainty means the team genuinely does not know which way the evidence will fall.

2. **Impact level.** The potential consequences—in terms of rework, schedule, cost, or quality degradation—if the hypothesis is invalidated. A high uncertainty combined with a high impact signals that a hypothesis demands urgent attention.

Both dimensions can be assessed using a 3-point or 5-point qualitative scale; assessments are made independently and relatively, not on absolute probability E. Silva, J. Melegati, Silveira, *et al.*, 2025.

2.2.3 Technical Plans

For hypotheses with significant uncertainty or impact, teams formulate a *technical plan*: a set of concrete actions whose goal is to (a) accept or refute the hypothesis, (b) reduce its uncertainty level, (c) reduce its potential impact, or (d) define criteria under which the decision can responsibly be deferred E. Silva, J. Melegati, Silveira, *et al.*, 2025. ArchHypo identifies the following plan strategies:

- **Architectural Spike.** A time-boxed, isolated experiment to explore a technology or design option without coupling the result to the production codebase. The spike produces information rather than deliverable functionality.

- **Tracer Bullet.** A thin, end-to-end implementation within the actual system, exercising all architectural layers to validate an integration or performance characteristic under realistic conditions.

- **Software Analytics.** Using monitoring, profiling, or usage data already collected from the running system to gather evidence about a hypothesis without additional development effort.

- **Modularity / Flexibility.** Restructuring the affected subsystem to reduce coupling or increase replaceability, thereby lowering the *impact* of the hypothesis rather than

resolving its uncertainty. This strategy is useful when full resolution is premature but protection against a bad outcome is warranted.

- **Guideline Implementation.** Tailoring the development process—for example, adopting a coding standard, a review protocol, or a testing policy—so that the team systematically generates evidence about the hypothesis as a by-product of normal work. This was the most frequently used strategy in the TSE evaluation E. Silva, J. Melegati, Silveira, *et al.*, 2025.
- **Architectural Trigger.** A condition-based deferral in which the team agrees to act on the hypothesis only when a specific observable event occurs (e.g., when the number of concurrent users exceeds a threshold, or when a performance regression appears in monitoring). Until the trigger fires, the hypothesis is deliberately deferred.
- **Agile Landing Zones.** Adopting architectural patterns that explicitly preserve flexibility, creating "landing zones" into which future decisions can be inserted without major disruption, regardless of which direction the evidence eventually points.

2.2.4 Lifecycle

Hypotheses progress through a defined lifecycle: they begin in an *open* state and transition to either *validated* or *invalidated* as evidence from technical plans accumulates. The responsible-moment principle—act when you have enough information, neither too early nor too late—guides when a hypothesis should be resolved rather than monitored further.

2.2.5 Empirical Evaluations

IEEE Software evaluation (Catch Solve startup). The first evaluation of ArchHypo in a real operating environment was reported in an IEEE Software article by Silva et al. E. Silva, J. Melegati, Wang, *et al.*, 2024. The setting was Catch Solve, a startup offering web testing and monitoring services based in Italy's South Tyrol region E. Silva, J. Melegati, Wang, *et al.*, 2024. Four architectural hypotheses were formulated and tracked over approximately ten months, covering quality attributes of availability, reusability of test templates, usability, and scalability of virtual machines. For example, the hypothesis *"The lack of redundancy can cause problems with application availability for the customers"* was assessed with a low impact and very low uncertainty, which led the team to defer investment in redundant infrastructure and instead set up an Architectural Trigger based on the number of new customers E. Silva, J. Melegati, Wang, *et al.*, 2024. The study showed that naming and tracking uncertainties explicitly helped the technical lead decide when to invest in architectural improvements versus when to wait, and validated the overall structure of the ArchHypo approach. A key insight from this evaluation was that explicit hypothesis management changed how the team prioritized technical work: rather than treating architectural uncertainty as urgent and tacit, it became visible, assessed, and deliberately managed within a defined lifecycle E. Silva, J. Melegati, Wang, *et al.*, 2024.

TSE evaluation (PropSEP at Jetsoft). The larger and more controlled evaluation, published in the IEEE Transactions on Software Engineering E. Silva, J. Melegati, Sil-

veira, *et al.*, 2025, examined the application of ArchHypo within Jetsoft, a Brazilian software development company, on the PropSEP project—the budget proposal system of the Planning and Management Secretariat of the State of São Paulo. PropSEP is mission-critical: the approved budget it processes amounts to approximately 317 billion BRL E. Silva, J. Melegati, Silveira, *et al.*, 2025. The evaluation ran across six sprints of seven business days each (May–August 2022) and involved eight participants (P1–P8), all holding computer science degrees with certifications in Oracle, WebSphere, or GeneXus. Ten architectural hypotheses were identified, spanning six quality attributes (performance, security, reliability, flexibility, usability, and team productivity), and seven distinct types of architectural practice were employed in the resulting technical plans E. Silva, J. Melegati, Silveira, *et al.*, 2025.

The study produced nine key findings:

1. Some hypotheses carried uncertainty that was monitored throughout the entire project duration without reaching full resolution, demonstrating that the technique accommodates long-horizon uncertainties E. Silva, J. Melegati, Silveira, *et al.*, 2025.

2. ArchHypo enabled the team to avoid upfront architectural design, distributing structural decisions across iterations as evidence became available E. Silva, J. Melegati, Silveira, *et al.*, 2025.

3. Participants reported improvements to the development process in terms of safety, transparency, and predictability E. Silva, J. Melegati, Silveira, *et al.*, 2025.

4. Hypotheses spanning different quality attributes were handled using qualitatively different technical strategies, illustrating the creativity the technique affords E. Silva, J. Melegati, Silveira, *et al.*, 2025.

5. Guideline Implementation (process tailoring) emerged as the most frequently employed strategy across all technical plans E. Silva, J. Melegati, Silveira, *et al.*, 2025.

6. The team collectively recognized that managing hypotheses improved the quality of their architectural decision-making E. Silva, J. Melegati, Silveira, *et al.*, 2025.

7. Post-release evidence pointed to a positive impact on product quality: no issues related to the managed hypotheses were detected after deployment E. Silva, J. Melegati, Silveira, *et al.*, 2025.

8. **Changes in the team's way of working constituted a significant adoption obstacle.** Mixing architectural and hypothesis-related tasks with functional development created estimation difficulties, and participants described the additional work as not feeling directly connected to product delivery E. Silva, J. Melegati, Silveira, *et al.*, 2025.

9. **All eight participants unanimously considered the technique hard to learn.** Specific difficulties included: mapping architectural concerns to the hypothesis format, formulating appropriate technical plan actions, articulating the assumptions behind a hypothesis, and understanding hypotheses as the primary driver of archi-

tectural action. No participant felt prepared to apply ArchHypo without continued expert support after the initial workshop E. Silva, J. Melegati, Silveira, *et al.*, 2025.

Findings 8 and 9 directly motivate the tooling contribution of this thesis: a tool that embeds ArchHypo scaffolding in the daily engineering workflow, reducing the cognitive overhead of manual application.

2.3 Internal Developer Portals and Backstage

The adoption of microservice architectures and the pursuit of team autonomy through practices such as Team Topologies have created significant complexity in the daily workflow of software engineers. In response, the industry has converged around the concept of *Internal Developer Portals* (IDPs) as a platform-engineering discipline aimed at reducing this complexity while preserving team independence Corrêa, 2025.

2.3.1 The Role of Internal Developer Portals (IDPs)

An Internal Developer Portal serves as the primary interface between developers and the underlying platform infrastructure. Unlike a simple wiki or a project-management tool, an IDP is an operational hub that aggregates tools, services, documentation, and data into a unified "pane of glass" Corrêa, 2025. Its primary goal is to reduce *cognitive load*: by abstracting the complexity of infrastructure and standardizing workflows, IDPs enable what practitioners call *Golden Paths*—recommended, supported, and automated paths for building and deploying software Corrêa, 2025.

This goal aligns directly with the theory of autonomous stream-aligned teams proposed by Skelton and Pais Skelton and Pais, 2019: if teams are to own full slices of a product without depending on a central gatekeeper, the platform must lower the barriers to acting on that ownership. An IDP operationalizes this by ensuring that the information and tooling a team needs—service catalog, documentation, CI/CD pipelines, monitoring dashboards, and domain-specific plugins—are accessible from a single environment, without context-switching.

Conceptually, IDPs sit at the intersection of two ideas that are highly relevant to this thesis. First, they embody the principle of *decentralized architecture*: rather than requiring teams to escalate architectural concerns to a central architecture group, a well-designed IDP makes it possible to encode architectural guidance, constraints, and tools directly into the developer workflow. Second, they are the natural delivery vehicle for hypothesis-engineering support: because ArchHypo must become part of the daily engineering cadence—not a separate off-sprint activity—embedding it in the portal where developers already spend their working time is strategically motivated.

2.3.2 Backstage: Open Platform for Internal Developer Portals

Backstage is an open-source framework for building Internal Developer Portals, originally developed at Spotify to manage the complexity arising from rapid growth and an expanding microservice estate. Spotify's internal version of Backstage serves as the central

nervous system for its infrastructure, used by over 2,700 engineers to manage more than 14,000 software components Spotify and Cloud Native Computing Foundation, 2024. Spotify open-sourced the framework in March 2020, and it was subsequently donated to the Cloud Native Computing Foundation (CNCF) as an Incubating project Spotify and Cloud Native Computing Foundation, 2024. As of 2025, more than 3,000 companies have adopted Backstage for their own portals Spotify and Cloud Native Computing Foundation, 2024.

Backstage is not a rigid application but rather a modular ecosystem driven by configuration. Its architecture rests on three layers: (1) the *Core Framework*, which manages essential utilities including the plugin loader, routing, authentication, and error handling; (2) the *Plugin Ecosystem*, built around a split-plugin model where frontend plugins handle views and UI components while backend plugins manage API services, data processing, and storage; and (3) *External Integrations*, through which backend plugins serve as gateways to external systems and APIs, allowing the portal to aggregate data from cloud providers, CI/CD tools, and other infrastructure Corrêa, 2025.

Crucially, the plugin architecture allows organizations to extend the portal's functionality by integrating custom tools directly into the developer's daily workflow, without forcing context-switching. It is precisely this extensibility that makes Backstage the appropriate delivery vehicle for Arch-Stage: a plugin can introduce a new class of first-class object—the architectural hypothesis—into the same environment where a developer already inspects service health, reviews documentation, and triggers CI/CD runs Corrêa, 2025.

The undergraduate capstone project of Corrêa Corrêa, 2025, supervised by the author of this thesis, demonstrated the technical feasibility of building such a plugin and validated the integration approach. That work established the initial version of Arch-Stage, which constitutes the artifact under empirical evaluation in this thesis.

2.4 Arch-Stage: The Initial Tool

Arch-Stage (originally named HypoStage) is an open-source Backstage plugin developed as an undergraduate capstone project by Pedro Henrique Mariano Corrêa, supervised by the author of this thesis and co-supervised by Prof. Paulo Meirelles Corrêa, 2025. Its purpose is to operationalize ArchHypo within an Internal Developer Portal (IDP), embedding hypothesis management into the engineering workflow.

The initial version of Arch-Stage, corresponding to the code up to commit `ceee509` in the `hypo-stage` repository,[1] provides the following capabilities:

- **Hypothesis management**: creating, editing, listing, and deleting architectural hypotheses, each linked to a Backstage catalog entity (e.g., a service, component, or system). Integration with the Backstage catalog means that hypotheses are automatically associated with the software entities they affect, without requiring separate registration.

[1] https://github.com/ArchHypo/hypo-stage

- **Uncertainty and impact assessment**: rating each hypothesis on 5-point Likert scales for both uncertainty level and impact level, with free-text rationale fields. The use of a 5-point scale rather than a binary or 3-point scale provides finer-grained prioritization while remaining tractable for teams without formal risk management backgrounds.
- **Technical planning**: attaching one or more technical plan items to a hypothesis, specifying action type (spike, tracer bullet, trigger, etc.), description, target date, expected outcome, and documentation links.
- **Visualization**: a chart plotting hypotheses on an uncertainty-vs-impact plane to support prioritization, making the relative urgency of hypotheses visible at a glance.
- **Status lifecycle**: tracking each hypothesis through the defined lifecycle states (open → validated or invalidated), preserving the history of status transitions to support retrospective analysis.

By integrating with Backstage, Arch-Stage leverages the portal's service catalog so that hypotheses are automatically associated with the software entities they affect. This design reduces context-switching: developers manage architectural hypotheses in the same environment where they consult documentation, inspect service health, and access CI/CD pipelines.

Known limitations. The initial tool design assumed familiarity with the ArchHypo vocabulary, which poses a barrier for teams new to hypothesis engineering. The case study in the capstone thesis reconstructed the Catch Solve startup scenario digitally but did not involve real-time use by a team. Furthermore, the tool lacked features for collaborative assessment (e.g., multiple raters per hypothesis) and for tracking the evolution of assessments over time. These gaps motivate the empirical evaluation and refinement effort in this thesis.

2.5 Deriving Architecture from Requirements: The Tooling Gap

The systematic mapping study by Souza, Moreira, and Goulão Souza *et al.*, 2019 examined the existing body of work on methods for deriving architectural models from requirements specifications. Starting from 2,575 candidate papers, the authors applied a PICOC (Population, Intervention, Comparison, Outcome, Context) search structure and a quality assessment checklist—composed of general criteria (weighted at 25%) and method-specific criteria (weighted at 75%)—to select 39 high-quality primary studies for detailed analysis. Their analysis employed the NIMSAD framework to characterize the structure of each reviewed method.

Among their key findings directly relevant to this thesis:

1. **Dependence on tacit knowledge.** Every reviewed method relied, to some degree, on the architect's experience and intuition. No method provided a fully automated

or guided derivation path accessible to less-experienced practitioners Souza *et al.*, 2019.

2. **Limited support for less-experienced practitioners.** Methods assumed a skilled architect capable of navigating trade-offs among quality attributes. Teams without senior architectural expertise had little actionable guidance Souza *et al.*, 2019.

3. **Weak or absent tool support.** Although some tools were identified—most notably QuaDAI and Monarch—they addressed only fragments of the derivation process and offered only partial coverage of the architectural reasoning cycle. The mapping found no comprehensive tool integrating hypothesis formulation, assessment, and tracking within a team's daily workflow Souza *et al.*, 2019.

4. **Insufficient evaluation.** Many proposed methods lacked rigorous empirical validation, relying on toy examples or author-run case studies rather than independent evaluations with real teams in production environments Souza *et al.*, 2019.

Souza et al. also noted that 76.9% of the 39 primary studies addressed functional requirements, with a significant proportion also treating non-functional requirements; yet the methods for handling quality attributes—precisely the domain where architectural uncertainty is most consequential—were among the least well supported by tools or empirical evidence Souza *et al.*, 2019.

These findings reinforce the motivation for this thesis on two fronts. First, they confirm the existence of a "tooling void" in which architectural management remains disconnected from the daily engineering workflow. Second, they underscore the need for empirical evaluations with real teams—precisely the kind of evaluation this thesis aims to provide for Arch-Stage and ArchHypo.

Chapter 3

Related Work

This chapter provides a contextual synthesis of literature relevant to the empirical evaluation of Arch-Stage. Rather than presenting a standalone systematic review, it draws on the author's earlier qualification work—which included a tertiary systematic mapping study on software architecture for startups and a multivocal literature review on centralization and decentralization of architectural decision-making—and repositions those findings as narrative context for the present thesis. The chapter also incorporates references from practitioner grey literature where relevant; such sources are explicitly identified as such, given that grey literature carries weaker evidential weight than peer-reviewed publications, but often captures industry trends and real organizational experience more rapidly than academic publication cycles allow.

3.1 Centralization and Decentralization of Architectural Decisions

A recurring theme in both academic and practitioner literature is the tension between centralizing architectural authority in a few experienced individuals and distributing it across development teams. Organizations pursuing fast flow and team autonomy increasingly favor decentralized models, yet several studies report that without explicit governance mechanisms, decentralization leads to inconsistent decisions and architectural drift.

The author's qualification examination explored this tension through two complementary lenses. The first chapter of the qualification presented a tertiary systematic mapping study surveying the existing landscape of secondary studies on software architecture for startup and scale-up organizations, identifying recurring themes in how early-stage teams approach architecture governance under resource and time constraints. The second chapter conducted a multivocal literature review that synthesized both academic publications and grey literature (practitioner blogs, conference talks, and industry reports), covering three interrelated perspectives: deriving software architecture from requirements, the emerging socio-technical architecture trend (which situates architectural decisions within organizational and team structures), and the tension between centralized and decentralized decision-making.

The multivocal review surfaced a framing that the qualification termed the "paradox of software architecture decision-making": industry advocates strongly for decentralized decisions to enable team autonomy and fast flow, yet the methods and tools available for architectural decision-making depend heavily on the tacit knowledge of experienced architects. Decentralization thus risks transferring decision responsibility to practitioners who lack the conceptual vocabulary or methodological support to exercise it effectively. The qualification committee, however, found this "paradox" framing ultimately unproductive as a primary research framing: the tension it described reflects a recognized design trade-off rather than a genuine logical contradiction, and building a thesis around resolving a paradox risked over-extending the scope of the contribution. This thesis consequently does not employ that framing; instead, it focuses on the concrete, tractable problem of providing tool support for one well-defined uncertainty management technique.

The third chapter of the qualification proposed a questionnaire-based research design to investigate how software engineering teams actually navigate architectural decisions in distributed settings. The proposed questionnaire targeted four dimensions: demographic and organizational context, knowledge of modern software architecture practices, collaboration and decision-making patterns within and across teams, and specific challenges experienced in decentralized architectural decision-making. While this research design would have yielded broad survey data, the qualification committee assessed the scope as too wide and the proposed SMS and multivocal review as insufficient as primary thesis contributions without a concrete artifact or evaluation at the center. The present thesis responds to that feedback by reorienting around the evaluation and refinement of Arch-Stage, a concrete tool artifact, rather than a survey of industry practice.

3.2 Socio-Technical Architecture

A significant body of work, both academic and practitioner-oriented, frames architectural decision-making not merely as a technical concern but as a *socio-technical* one—shaped by the organizational structure, communication patterns, and cognitive constraints of the teams involved. This framing is directly relevant to the motivation for Arch-Stage, because tool-embedded scaffolding does not only lower the technical cost of applying ArchHypo: it also changes the social dynamics of how architectural uncertainty is surfaced and addressed within a team.

3.2.1 Conway's Law and Its Implications

The foundational observation underlying the socio-technical framing is Conway's Law Conway, 1968: "Any organization that designs a system (defined broadly) will produce a design whose structure is a copy of the organization's communication structure." Conway's Law implies that architectural decisions cannot be analyzed in isolation from the organizational and team structure in which they are made. Team boundaries become architectural boundaries; communication bottlenecks manifest as coupling in the system design.

Skelton and Pais Skelton and Pais, 2019 extended this insight into an actionable framework with the concept of the *Inverse Conway Maneuver*: rather than allowing the organization's structure to passively determine the architecture, teams should deliberately

design team topologies to produce the target architecture. In practical terms, this means that architectural governance—including how uncertainty is managed and who is empowered to make decisions—must be considered a design decision with organizational as well as technical consequences.

3.2.2 Socio-Technical Architecture as an Industry Trend

The InfoQ Software Architecture and Design Trends Report for 2024 Betts *et al.*, 2024 identified *socio-technical architecture* as a distinct "Early Adopter" trend, explicitly describing it as replacing the earlier framing of "architecture as a team sport." The report notes that the trend encompasses two related ideas: on one side, the evolving role of architects as mentors and technical leaders rather than sole decision-makers; on the other, the growing recognition—driven in part by the dissemination of Team Topologies Skelton and Pais, 2019—that the design of a system must consider all the people who build, support, and maintain it Betts *et al.*, 2024.

It is important to note that InfoQ is a practitioner-oriented publication, not a peer-reviewed academic venue. The Trends Report reflects the editorial judgment of an experienced editorial board drawing on community submissions, conference talks, and practitioner case studies. As grey literature, its findings carry weaker evidential weight than systematic empirical studies; nonetheless, they are valuable as an indicator of what concepts are gaining traction in industrial practice and what challenges practitioners are actively grappling with Betts *et al.*, 2024.

In a related InfoQ article, Hunter and Stojmilova Hunter and Stojmilova, 2025 described a transformation journey at a software organization adopting Team Topologies and decentralized architectural decision-making. Their account illustrates several socio-technical dynamics that are directly relevant to this thesis: the initial disbelief of teams that they are actually empowered to make architectural decisions, the flood of decisions that follows once empowerment is accepted, the consequences teams face when decisions have unforeseen cross-boundary impacts, and the eventual self-correction as teams develop peer accountability mechanisms. All four phases are driven by social factors at least as much as technical ones Hunter and Stojmilova, 2025.

A further grey-literature source relevant to this work is the InfoQ article by Tibbitts Tibbitts, 2025, which argues that "distributed architecture is more social than technical." Tibbitts draws on Andrew Harmel-Law's concept of the Architecture Advice Process Tibbitts, 2025—in which any team can make any architectural decision as long as it has sought advice from affected parties and documented the decision in an ADR—as a mechanism for replacing permission-seeking with advice-seeking. The social shift from "who approves this?" to "who should I talk to before we do this?" is presented as a cultural unlock that enables decentralized governance without sacrificing coherence Tibbitts, 2025.

3.2.3 Centralization vs. Decentralization: The Core Tension

The tension between centralized architectural authority and distributed team autonomy can be framed as follows. Centralized approaches, typified by the "ivory tower" architect role, ensure consistency and coherence but create bottlenecks, reduce team ownership,

and can slow delivery as the central authority becomes a gating dependency Hunter and Stojmilova, 2025. Decentralized approaches empower teams and support fast flow but risk architectural drift, duplicated effort, and decisions made without the contextual awareness that crosses team boundaries.

Practitioner accounts such as those of Hunter and Stojmilova Hunter and Stojmilova, 2025 suggest that the resolution lies not in choosing one extreme but in providing the right scaffolding: architectural principles to guide decisions, ADRs to document them, advisory forums to share them, and—crucially for this thesis—tools embedded in the daily workflow to make the practice tractable for all team members, not only experienced architects. This is precisely the gap that Arch-Stage is designed to fill. Where existing approaches to decentralized governance focus primarily on the *documentation* and *communication* of settled decisions (ADRs, advice forums), Arch-Stage addresses the *pre-decision* space: managing the uncertainty that precedes and motivates architectural choices.

3.3 Tool Support for Architectural Decision-Making

Several tools and platforms have been proposed to support architectural decision-making:

- **ADR tools** (e.g., adr-tools, Log4brains) focus on recording and versioning Architecture Decision Records (ADRs). They capture the *what* and *why* of decisions but do not model uncertainty, impact, or technical plans for hypotheses that are not yet decisions. ADRs document settled choices; they are not designed to manage the pre-decision state of uncertainty that ArchHypo addresses.

- **Architecture analysis tools** (e.g., ATAM-based workshops, trade-off analysis methods) provide structured evaluation of architectural alternatives but are typically heavyweight, workshop-based activities rather than continuous, team-embedded processes. Their value lies in one-shot or periodic analysis sessions, not in the ongoing, iterative management of uncertainty throughout a product's lifecycle.

- **Internal Developer Portals** such as Backstage, Port, and Cortex offer a catalog-centric view of services and infrastructure. While they improve discoverability and documentation, none of these platforms natively supports hypothesis engineering or architectural uncertainty management. They provide the integration surface that Arch-Stage exploits, but leave the hypothesis engineering layer entirely absent.

Arch-Stage differentiates itself by being explicitly designed around the ArchHypo lifecycle: from hypothesis elicitation through assessment, technical planning, and resolution. Its integration with Backstage places this lifecycle within the developer's daily tooling environment. No existing tool, to the best of the author's knowledge, provides this combination.

3.4 Empirical Studies of Architectural Practices in Industry

Industry-focused empirical studies on architectural practices remain relatively scarce compared with tool proposals and method descriptions. Among the most relevant:

- The ArchHypo evaluations by Silva et al. E. Silva, J. Melegati, Silveira, *et al.*, 2025E. Silva, J. Melegati, Wang, *et al.*, 2024 constitute the closest antecedents to this work. They demonstrated the value of hypothesis engineering in two distinct settings—a web-testing startup and a mission-critical government budget system—but relied entirely on manual application with no tool support. This thesis introduces the first evaluation of ArchHypo in a tool-mediated setting.
- Studies on architectural governance in agile organizations (e.g., the work on SAFe and Spotify-model variants) document how companies structure roles and ceremonies for architecture, but rarely examine explicit uncertainty-management techniques.
- The multivocal review conducted during the author's qualification surfaced practitioner reports (blog posts, conference talks, white papers) describing ad-hoc approaches to architectural uncertainty. These reports corroborate the need for structured methods but, being grey literature, carry weaker evidential value.

This thesis contributes to the body of empirical evidence by evaluating a tool-supported application of ArchHypo with multiple teams, addressing a gap in both the ArchHypo literature (which has not yet studied tool-mediated use) and the broader architectural-practices literature (which lacks studies of hypothesis-engineering tools in scaled settings).

Chapter 4

Research Design

This chapter describes and justifies the research design adopted in this work. It covers the adopted research strategy along with the conceptual framework created to fullfill the research questions. Moreover, it covers the study context and case selection, the researcher's role, the data collection procedures, and the criteria adopted to establish the trustworthiness of the findings. The chapter concludes with notes on the ethical considerations that governed the study.

4.1 Research Strategy

The objective of this work was to empirically evaluate and refine Arch-Stage, a support tool for the ArchHypo technique, by applying it with multiple software engineering teams in a real-world organizational setting. Answering the research questions posed in Chapter 1 requires understanding not only *what* outcomes the tool produces, but also *how* teams interact with it, *why* certain difficulties arise, and *which* adjustments improve its fit with existing practices. Following the design framework proposed by Robson and McCartan, 2016, the purpose of this work combined with its research questions clearly pointed out the need for a **flexible research design**.

Within flexible designs, the study follows a **multiple case study strategy** as defined by Robson and McCartan, 2016; Yin, 2009. A case study is defined by them as an empirical investigation of a particular contemporary phenomenon within its real-life context, using multiple sources of evidence and where the boundary between phenomenon and context is not clearly distinct. This definition fits the present work precisely: the phenomenon under study is the use of Arch-Stage by a software engineering team, and its context — the organizational, technical, and social environment in which the team operates — is both relevant and inseparable from the phenomenon itself. A purely experimental design, which would require controlled manipulation of variables and prespecification of the design before data collection, is not appropriate here. The number of teams is small, the settings are naturalistic and non-manipulable, and the research purpose calls for rich, process-oriented accounts rather than statistical inference Robson and McCartan, 2016.

The study involves **two cases**, each consisting of a distinct software engineering team.

Using multiple cases rather than a single case follows the logic of *analytic (or theoretical) generalization* from Robson and McCartan, 2016; Yin, 2009: the two cases are not selected to constitute a statistical sample of teams, but to provide complementary and contrasting instances through which patterns of tool use, difficulties, and refinements can be examined and cross-validated. As Robson and McCartan, 2016 argues, multiple case studies are analogous to multiple experiments where each case either replicates findings or provides theoretically predicted contrast — both strengthening the analytic basis for generalization.

Beyond its descriptive and explanatory purposes, the study also has an **evaluative purpose**: it seeks to assess the worth and fitness of Arch-Stage as a practical tool for managing architectural uncertainty in software engineering teams. Robson and McCartan, 2016 distinguish between *formative evaluation*, which aims at understanding and improving an ongoing program or intervention, and *summative evaluation*, which renders a final verdict on overall effectiveness. This study is explicitly formative: data collected during the observation period are used to identify difficulties and trigger tool refinements in near real-time, rather than to produce a retrospective pass-or-fail judgment.

Finally, the iterative loop through which the researcher observes difficulties, implements refinements to the tool, deploys an updated version, and resumes observation incorporates a core element of **action research** as described by Robson and McCartan, 2016: a deliberate cycle of plan, act, observe, and reflect, with the explicit intent of producing practical change alongside research knowledge. This does not transform the study into a full action research project — the primary goal remains the production of knowledge about tool use and architectural uncertainty, not organizational change more broadly — but the iterative refinement component follows the same logic and must be acknowledged as part of the design.

In summary, the research strategy can be formally described as a **flexible, multiple case study design with formative evaluation purpose and action research elements**, following the taxonomy from Robson and McCartan, 2016.

4.2 Conceptual Research Framework

The conceptual research framework organizes the key manufactures of this work and their relationships in a way that is consistent with the research questions and guides the data collection and analysis as proposed by Robson and McCartan, 2016. Figure 4.1 describes the framework.

The central piece is the *Arch-Stage Tool Usage*, understood as the practices through which team members appropriate and practice the ArchHypo technique via Arch-Stage: creating and editing architectural hypotheses, assessing uncertainty levels and impact, defining and tracking technical plans, and revisiting hypotheses over successive sprint cycles K. Silva, J. Melegati, *et al.*, 2025. The tool usage is conditioned by two sets of contextual factors: the *team context* (composition, experience profile, scope of work, and familiarity with architectural concerns) and the *organizational context* (the company's development platform, the role of Backstage as the delivery portal, and the scaled product being developed). The tool usage produces two observable streams: the *artifacts* recorded in Arch-Stage (hypotheses, assessments, technical plans, and their evolution over time)

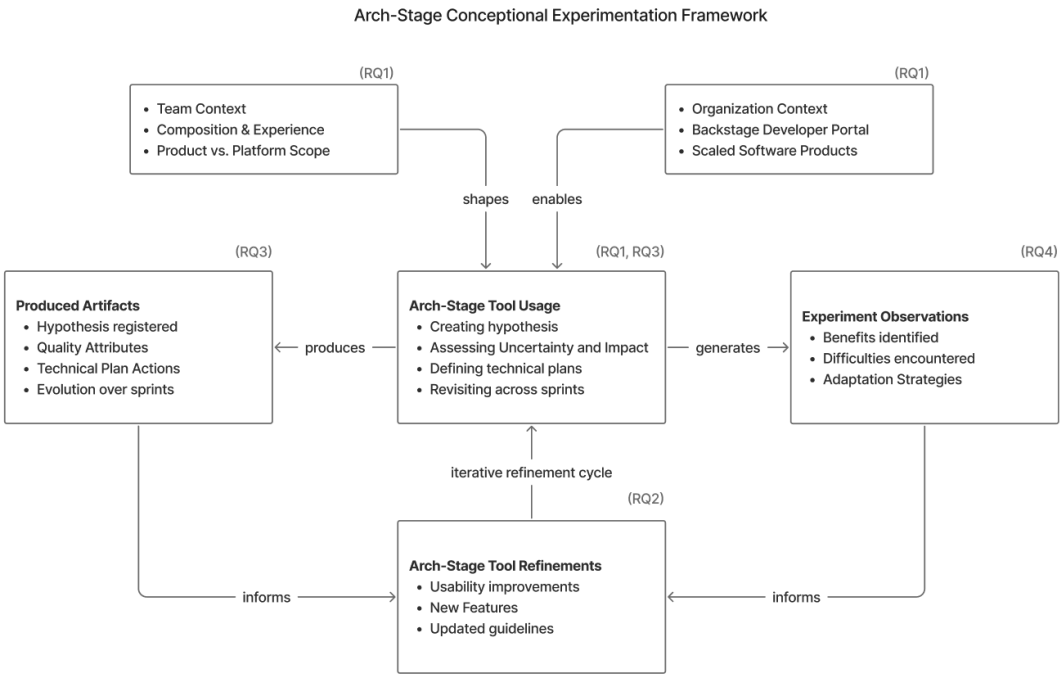

Figure 4.1: *Conceptual framework of the study.*

and the *experiment observations* of team members (benefits, difficulties, and adaptation strategies). These streams feed the *Arch-Stage Tool Refinements* — changes made to Arch-Stage and its usage guidelines — which in turn modify subsequent tool usage, forming an iterative cycle. This framework is consistent with the formative evaluation and action research elements of the design, and directly links each element to the research questions (RQ1–RQ4) stated in Chapter 1.

4.3 Research Context

The study was conducted within a technology organization that operates large-scale software products across Latin America and employs over 500 software engineers. The organization is an intensive user of Backstage, an open-source developer portal, which serves as the central platform for internal tooling, documentation, and engineering workflows. The availability of this platform was an enabling condition for the study: Arch-Stage was deployed as a Backstage plugin, so its adoption is conditioned by the availability of that platform. The Arch-Stage tool was deployed directly in the company's production environment, integrated with the same catalog and workflow interface that engineers use daily, reducing adoption friction considerably. This allowed the researcher to observe the tool usage in a naturalistic setting, with minimal interference from the researcher.

4.3.1 Case Selection

Two teams were selected as cases. Following Robson and McCartan, 2016's discussion of purposive sampling in flexible designs, the cases were not chosen to be statistically representative of a population of teams, but to provide *complementary contrast*: teams with

different compositions, scopes, and relationships to the broader system, so that differences and similarities in tool use can be analytically compared.

Team 1 — Product Team. A team of 12 software engineers (5 senior, 7 mid-level) responsible for developing new product features with direct impact on the company's business objectives and end-user experience. This team works on a bounded product domain, with close proximity to business requirements and user feedback. Its architectural concerns are primarily about supporting new features reliably and at scale while preserving existing quality attributes.

Team 2 — Platform Team. A team of 4 software engineers (2 senior, 2 mid-level) responsible for developing and maintaining broad-scope internal tooling that supports software development across the entire organization. This team's architectural decisions affect how all engineers in the company build, deploy, and operate software. Its architectural concerns are therefore more horizontal, with higher impact on organizational engineering practices and greater uncertainty about usage patterns and requirements.

The two cases represent a *literal replication* in so far as both teams face the same intervention (use of Arch-Stage and ArchHypo) and a *theoretical replication* in that their different scopes and compositions should, theoretically, lead to different experiences of architectural uncertainty and different challenges in adopting a hypothesis-driven approach Robson and McCartan, 2016; Yin, 2009. This contrast strengthens the analytic basis for identifying which findings are context-independent and which are conditioned by team type.

4.4 Researcher's Role

Throughout the study, the researcher occupied the dual role of **participant observer and facilitator**, in the terms described by Robson and McCartan, 2016. The researcher is an employee of the organization in which the study was conducted, which makes this an instance of *insider research*. This position provided privileged access to the teams, their work context, and their daily practices, which would have been substantially harder to obtain as an external investigator. It also allowed the researcher to introduce Arch-Stage into the company's infrastructure in a credible and practically feasible way.

Robson and McCartan, 2016 acknowledge that insider research raises specific concerns regarding bias and the potential conflation of the researcher's own perspective with those of participants. These concerns are addressed in Section 4.8. It is important to note that, while the researcher served as facilitator during training sessions and provided assistance on request throughout the observation period, he did not direct the teams in *which* architectural uncertainties to address or *how* to formulate specific hypotheses. The content of the work produced by the teams in Arch-Stage reflects their own judgments and priorities.

4.5 Procedure

4.5.1 Training and Introduction

Before the observation period began, each team participated in a one-hour training session led by the researcher. The session covered:

(i) The conceptual foundations of hypotheses engineering and the ArchHypo technique, including the distinction between architectural hypotheses, uncertainty levels, impact assessment, and technical plans.

(ii) The context and motivation of the research, including the broader challenge of making architectural uncertainties explicit in teams working on scaled products.

(iii) A live demonstration of Arch-Stage running in the company's Backstage instance, illustrating how to create and manage hypotheses within the tool. Teams were given written communication prior to the meeting (Appendix A.4) — including an illustration of the ArchHypo big picture (Figure 4.2) — so that they could arrive at the session with some prior exposure to the key concepts.

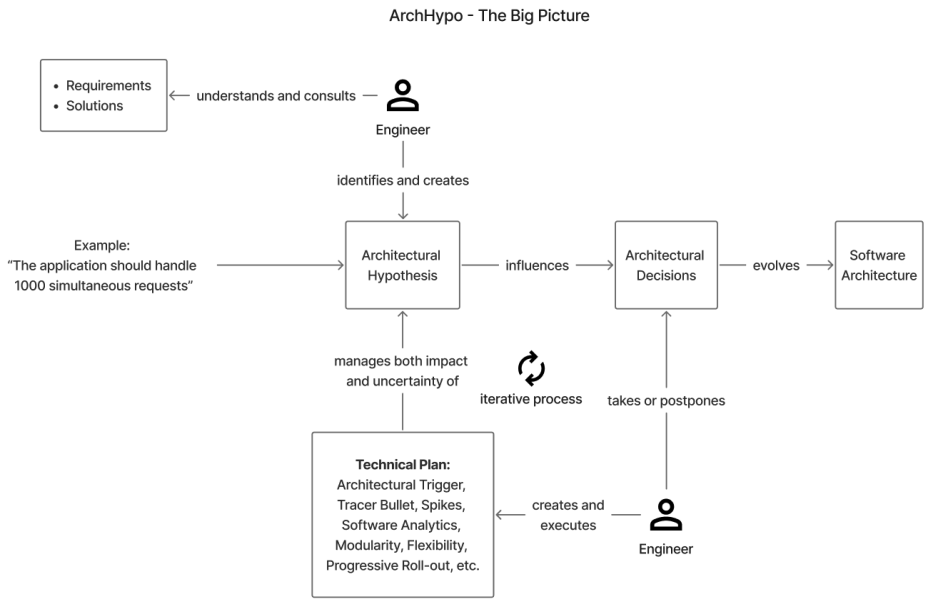

Figure 4.2: *Overview of the ArchHypo technique showing the iterative cycle through which engineers identify architectural hypotheses, assess their uncertainty and impact, define technical plans, and take or postpone architectural decisions. Source: adapted from K. Silva, J. Melegati, et al., 2025.*

4.5.2 Observation Period

Following the training session, each team entered a two-sprint observation period of approximately four weeks, during which they were invited to use Arch-Stage to register and manage any architectural uncertainties that arose in the course of their regular development work. No constraints were imposed on the type, number, or formulation of hypotheses, nor on when to create them during the sprints. It is intrisic to the nature of the

organization in which the experience was conducted that the teams are free to adjust their agile practices to their own needs. Thefore, in contrast to K. Silva, J. Melegati, *et al.*, 2025, the teams were not required to create hypotheses during the sprints, but rather to use the tools whenever they deemed necessary. The objective was to observe tool use in naturalistic conditions, with minimal researcher interference in the content of the teams' work.

During this period, the researcher remained available to both teams for support on request. Support was of two distinct types:

1. **Conceptual support.** When a team member found it difficult to formulate a hypothesis or to assess its uncertainty and impact within the ArchHypo framework, the researcher provided clarification or facilitated a brief discussion. Interactions of this type were noted as observations about the conceptual difficulty of the technique.

2. **Tool support.** When a team member encountered a difficulty that originated from the tool itself — a usability problem, a missing feature, or a mismatch between the tool's behavior and the expected workflow — the researcher logged this as a tool improvement candidate and, as a matter of priority, implemented and deployed an updated version of Arch-Stage as quickly as practicable. This was done to ensure that tool limitations would not persistently obstruct teams' ability to practice ArchHypo, and to allow subsequent observations to reflect a more refined version of the tool.

This second type of support operationalizes the *action research cycle* within the study: difficulty observed → refinement designed → updated tool deployed → use and observation resumed. Each refinement cycle is documented in Chapter 6.

4.6 Data Collection

Following the multiple data source approach recommended for case studies by Robson and McCartan, 2016; Yin, 2009, data were collected through three complementary channels:

4.6.1 Participant Observation

The researcher's continuous presence in the organization and involvement in both teams' environments provided an ongoing stream of observational data. Observations covered:

(i) Team members' reactions and difficulties during and after the training session;

(ii) Patterns of use (or non-use) of Arch-Stage during the sprint;

(iii) Interactions in which team members requested conceptual or tool support;

(iv) Informal commentary about the fit of ArchHypo with existing practices.

Following Robson and McCartan, 2016's guidance on participant observation, field notes were recorded as close as possible in time to the observed events, and the researcher maintained a reflexive awareness of his dual role as insider and investigator.

4.6.2 Artifact Analysis

The hypotheses, assessments, and technical plans created by each team in Arch-Stage constitute a documentary record of how they used the tool over the observation period. These artifacts were collected at the end of the observation period and analysed to characterize:

(i) The number and types of hypotheses created;

(ii) The quality attributes and sources of uncertainty they addressed;

(iii) How hypotheses were assessed and what technical plan actions were defined;

(iv) How hypotheses evolved (created, refined, or closed) across the two sprints.

The content analysis of documentary record is a recognized data collection method in flexible case study designs by Robson and McCartan, 2016.

4.6.3 Open-Ended Active Listening Sessions

In addition to informal interactions arising from support requests, the researcher conducted brief open-ended active listening sessions with team members at the end of the observation period, inviting them to reflect on their experience of using Arch-Stage and ArchHypo. These conversations followed a semi-structured format, covering the perceived usefulness of the technique, the difficulties encountered, the changes they would suggest to the tool or its integration into their workflow, and their overall assessment of the value of making architectural uncertainties explicit in this way. Semi-structured or focused interviews are a central method for flexible design research and are particularly appropriate when the researcher seeks to capture participants' perspectives and interpretations rather than confirming predetermined hypotheses as described by Robson and McCartan, 2016.

4.7 Data Analysis

Data from the three collection sources were analysed using **thematic coding analysis**, following the procedure described by Robson and McCartan, 2016 for qualitative data in flexible designs Robson and McCartan, 2016. The process proceeded in three stages.

First, all field notes, artifact summaries, and active listening session records were reviewed and coded inductively, generating an initial set of descriptive codes capturing recurring observations and themes. Second, codes were grouped into higher-level categories corresponding to the constructs in the conceptual framework: *patterns of tool use*, *types of architectural uncertainty managed*, *perceived benefits*, *perceived challenges*, and *candidate tool refinements*. Third, patterns within and across the two cases were compared to identify what was common to both teams and what appeared specific to the product or platform context.

The artifact data from Arch-Stage were also summarized descriptively (number of hypotheses per team, quality attributes addressed, evolution across sprints) to complement and illustrate the qualitative findings. The use of these summary statistics is consistent

with Robson and McCartan, 2016's observation that flexible designs can include small amounts of quantitative data without becoming multistrategy designs.

4.8 Establishing Trustworthiness

The trustworthiness of findings from flexible design research is discussed by Robson and McCartan, 2016 in terms of *credibility*, *transferability*, *dependability*, and *confirmability* Robson and McCartan, 2016. Each is addressed below.

Credibility — the internal validity of the findings — was supported through:

(i) Prolonged engagement with both teams over four weeks of real work in their natural environment;

(ii) Multiple data sources (observation, artifact analysis, active listening sessions), providing triangulation of evidence;

(iii) Member checking, in which interpretations derived from observations and conversations were shared with team members for verification and correction.

Transferability — the degree to which the findings may inform other settings — is constrained by the specifics of the organizational context (a large technology company in Latin America with an established Backstage platform) and the tool (Arch-Stage). The study does not claim statistical generalization. Rather, it offers analytic generalization Robson and McCartan, 2016; Yin, 2009: the patterns of use, difficulties, and refinements documented here provide a theoretically informed account that researchers and practitioners can assess for relevance to their own contexts.

Dependability — the reliability of the research process — was addressed by maintaining an audit trail: field notes were dated and linked to specific events, artifact data were extracted and stored systematically, and each tool refinement was documented with its triggering observation and the corresponding commit in the Arch-Stage repository.

Confirmability — the degree to which findings reflect the data rather than researcher bias — was addressed by:

(i) Explicitly distinguishing between the researcher's facilitation role (which involved helping teams understand the technique) and his investigator role (which required non-directive observation of how they used it);

(ii) Seeking disconfirming evidence, that is, noting instances where teams found value in the tool or the technique as readily as instances of difficulty;

(iii) Grounding all claims in specific observations, artifacts, or conversation excerpts, reported in Chapter 5.

4.9 Ethical Considerations

The study raises ethical considerations associated with insider research, as discussed by Robson and McCartan, 2016. Participation was voluntary: team members were informed

of the research objectives, were free to use Arch-Stage as much or as little as they chose, and were invited but not required to engage in active listening sessions. No personal performance data were collected or reported; all analysis pertains to team-level patterns and tool-level observations. The organization in which the study was conducted and the teams involved are anonymized in reporting. As no sensitive personal data were collected, no ethical review was required for this study.

The dual role of the researcher as organizational employee and academic investigator is a potential source of role conflict. This was managed by maintaining a clear separation between the researcher's responsibilities to the organization (deploying and supporting a useful tool) and his responsibilities as a researcher (reporting findings honestly, including negative ones). The research was conducted with the knowledge and informal consent of the relevant engineering leadership in the organization.

Chapter 5

Results

Chapter 6

Refinements

Chapter 7

Conclusion

Appendix A

Research Instruments

A.1 Questionnaire

A.2 Interview Guide

A.3 Informed Consent Form

A.4 Training Communication

The following message was sent to teams before the training session that introduced the ArchHypo experiment and the Arch-Stage plugin in Backstage.

> **New Plugin Experiment in Backstage: Hypo Stage — Practicing the ArchHypo Framework**
>
> Deciding when to make (or delay) a software architecture decision is one of the toughest daily challenges in engineering, regardless of the project:
>
> - Deciding too early can lock the system into the wrong path.
> - Deciding too late can lead to costly rework and technical debt.
>
> In the middle of this lies something that often remains implicit: uncertainty. We deal with requirements that are not yet clear or solutions we are not sure will scale or integrate well. This uncertainty is rarely made explicit, and decisions often end up being built on unvalidated premises.
>
> To evolve past this dilemma, I am proposing the application of a framework to help us identify, evaluate, and manage architectural uncertainties in a structured way. We will put this into practice using the Hypo Stage plugin, which I helped develop as part of an open-source project. It is already available in our Backstage (in production and under a feature flag for our experiment).
>
> This plugin serves as a tool to practice the ArchHypo (Architectural Hy-

potheses) framework: a lightweight approach that applies hypothesis-driven engineering to represent uncertainties impacting software architecture. It helps us plan actions to reduce uncertainty or impact before "locking in" a decision.

What ArchHypo proposes, in short:

Architectural Hypotheses: Falsifiable statements expressing team beliefs about requirements (e.g., quality attributes) or current/candidate solutions. The focus is on uncertainties that affect architectural decisions, not just business hypotheses.

Evaluation: Each hypothesis is assessed based on its uncertainty level (how far we are from proving it true/false) and impact (the consequences and cost of being wrong). This helps prioritize and choose treatment tactics.

Technical Plan: For each hypothesis, the team defines actions to either reduce uncertainty (e.g., spikes, experiments, architectural triggers, metrics) or reduce impact (e.g., modularity, flexibility, isolation of the affected part), allowing us to delay decisions more safely.

Iterative Cycle: Identify, evaluate, plan, execute, and reassess. The process is continuous; as new uncertainties emerge, plans are adjusted over time.

The Hypo Stage plugin in Backstage allows you to register hypotheses, link them to quality attributes, record uncertainty/impact assessments, and maintain technical plans with dates and progress tracking — all integrated into our catalog and workflow.

Want to better understand the problem, the framework, and how to use the plugin?

This Week Day, Calendar Date, at HH:MM during the Meeting Agenda, I will be leading a session on this topic. We will discuss why finding the right moment for architectural decisions is so difficult and how ArchHypo can help.

I am looking forward to seeing you there to discuss this experiment and how to use Hypo Stage in our daily routine.

References

[Bass *et al.* 2021] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. 4th ed. Addison-Wesley, 2021 (cit. on p. 7).

[Betts *et al.* 2024] Thomas Betts, Blanca García Gil, Eran Stiller, Daniel Bryant, and Rafal Gancarz. *InfoQ Software Architecture and Design Trends Report — 2024*. https://www.infoq.com/articles/architecture-trends-2024/. Accessed: 23 March 2026. 2024 (cit. on pp. 1, 2, 4, 6, 19).

[Conway 1968] Melvin E. Conway. "How do committees invent?" *Datamation* 14.4 (1968), pp. 28–31 (cit. on p. 18).

[Corrêa 2025] Pedro Henrique Mariano Corrêa. *ArchHypo Tooling: Enabling Practical Hypothesis Engineering for Software Architecture*. Capstone Project Report (Bachelor), Institute of Mathematics and Statistics, University of São Paulo. Supervised by José G. Lima Neto; co-supervised by Paulo Meirelles. 2025 (cit. on pp. 3–5, 12, 13).

[Foote and Yoder 1999] Brian Foote and Joseph Yoder. "Big ball of mud". *Pattern Languages of Program Design* 4 (1999), pp. 654–692 (cit. on p. 8).

[Hunter and Stojmilova 2025] Peter Hunter and Elena Stojmilova. "Empowering teams: decentralizing architectural decision-making". *InfoQ* (Nov. 2025). Grey literature — practitioner article. URL: https://www.infoq.com/articles/empowering-decentralizing-architectural-decision-making/ (cit. on pp. 19, 20).

[Perry and Wolf 1992] Dewayne E. Perry and Alexander L. Wolf. "Foundations for the study of software architecture". *ACM SIGSOFT Software Engineering Notes* 17.4 (1992), pp. 40–52. DOI: 10.1145/141874.141884 (cit. on pp. 1, 7).

[Richards and Ford 2020] Mark Richards and Neal Ford. *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media, 2020. ISBN: 9781492043454 (cit. on p. 7).

[Robson and McCartan 2016] Colin Robson and Kieran McCartan. *Real World Research: A Resource for Users of Social Research Methods in Applied Settings*. 4th. John Wiley & Sons, 2016. ISBN: 978-1-118-74523-6 (cit. on pp. 23–26, 28–30).

[E. Silva, J. Melegati, Silveira, *et al.* 2025] Eduardo Silva, Jorge Melegati, Moises Silveira, *et al.* "ArchHypo: managing software architecture uncertainty using hypotheses engineering". *IEEE Transactions on Software Engineering* 51.2 (2025), pp. 481–499. doi: 10.1109/TSE.2024.3520477 (cit. on pp. 8–12, 21).

[E. Silva, J. Melegati, Wang, *et al.* 2024] Eduardo Silva, Jorge Melegati, Xiaofeng Wang, and Paulo Meirelles. "Using hypotheses to manage technical uncertainty and architecture evolution in a software start-up". *IEEE Software* 41.4 (2024), pp. 78–87. doi: 10.1109/MS.2024.3383628 (cit. on pp. 8, 10, 21).

[K. Silva, J. Melegati, *et al.* 2025] Kelson Silva, Jorge Melegati, *et al.* "Archhypo: managing software architecture uncertainty using hypotheses engineering". *IEEE Transactions on Software Engineering* 51.2 (2025), pp. 430–446. doi: 10.1109/TSE.2024.3520477 (cit. on pp. 1–4, 24, 27, 28).

[K. Silva, J. A. Melegati, *et al.* 2024] Kelson Silva, Jorge A. Melegati, Xiaofeng Wang, Mauricio Ferreira, and Eduardo Guerra. "Using hypotheses to manage technical uncertainty and architecture evolution in a software start-up". *IEEE Software* 41.4 (2024), pp. 7–13. doi: 10.1109/MS.2024.3383628 (cit. on pp. 1–4).

[Skelton and Pais 2019] Matthew Skelton and Manuel Pais. *Team Topologies: Organizing Business and Technology Teams for Fast Flow.* IT Revolution Press, 2019 (cit. on pp. 7, 8, 12, 18, 19).

[Souza *et al.* 2019] Eric Souza, Ana Moreira, and Miguel Goulão. "Deriving architectural models from requirements specifications: a systematic mapping study". *Information and Software Technology* 109 (2019), pp. 26–39. doi: 10.1016/j.infsof.2019.01.004 (cit. on pp. 1–4, 6, 14, 15).

[Spotify and Cloud Native Computing Foundation 2024] Spotify and Cloud Native Computing Foundation. *Backstage: An Open Source Framework for Building Developer Portals.* https://backstage.io. Open-source project; over 3,000 companies adopted as of 2025. CNCF Incubating Project. 2024 (cit. on p. 13).

[Tibbitts 2025] Lindsey Tibbitts. "Decentralized architecture needs more than autonomy". *InfoQ* (June 2025). Grey literature — practitioner article. url: https://www.infoq.com/articles/decentralized-architecture-advice-process/ (cit. on p. 19).

[Yin 2009] Robert K. Yin. *Case Study Research: Design and Methods.* 4th. Thousand Oaks, CA: SAGE Publications, 2009 (cit. on pp. 23, 24, 26, 28, 30).